



University of
Nottingham
UK | CHINA | MALAYSIA

Lecture 7: Regression and Classification

COMP1037 Fundamentals of Artificial Intelligence

Xinan Chen

University of Nottingham Ningbo China

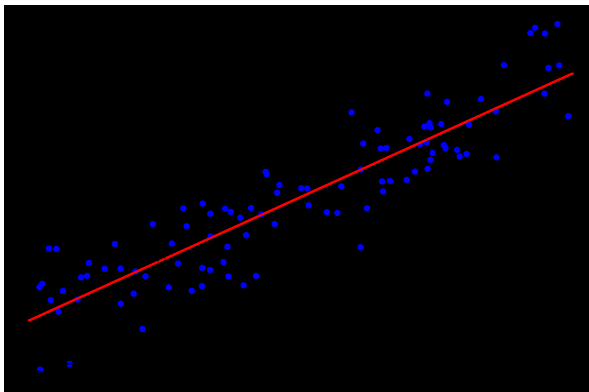
April 6, 2025

Outline |

1. Regression
 - Polynomial Regression
2. Classification
 - Logistic Regression
 - Naive Bayes
3. Overfitting and Underfitting

A Simple Example of Linear Regression |

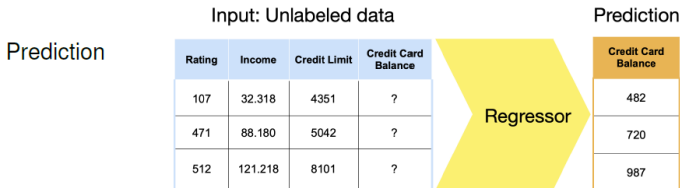
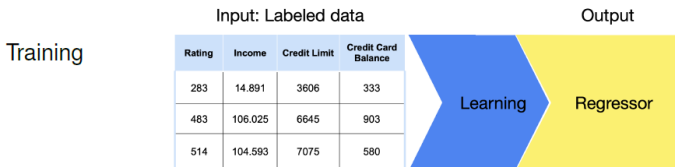
$$y = kx + b$$



Linear Regression |

- Learn **linear combination of features** for predicting the target variable.
- Find best parameters of the model by training with respect to a **loss function**.

$$\text{Credit Balance} = \theta_0 + \theta_1 \text{Rating} + \theta_2 \text{Income} + \theta_3 \text{CreditLimit}$$

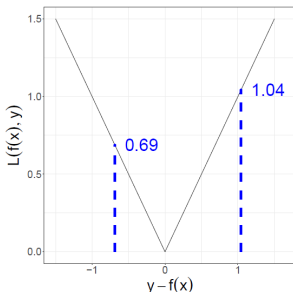


L1 and L2 Loss |

Definition

Loss can be characterized as a function of **residuals**:

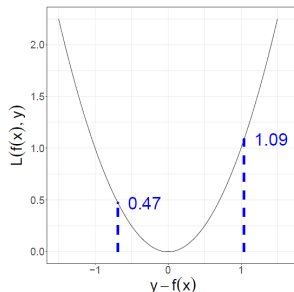
$$r = y - f(x)$$



L1 penalizes the **absolute** value of residuals

$$L(r) = |r|$$

Robust to outliers



L2 penalizes the **quadratic** value of residuals

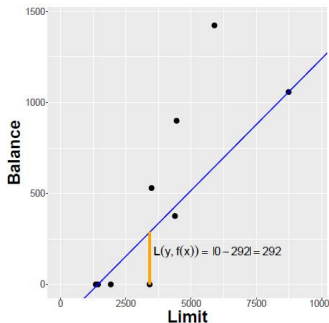
$$L(r) = r^2$$

Easier to optimize

L1 and L2 Loss |

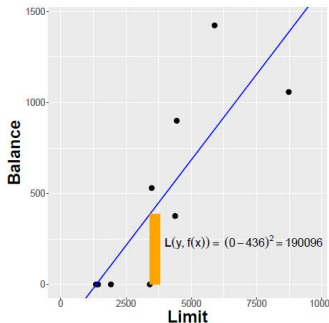
L1 Loss

Not differentiable at $r = 0$.
Optimal parameters are computed **numerically**.



L2 Loss

Differentiable everywhere.
Optimal parameters can be computed **analytically or numerically**.



L2 Loss |

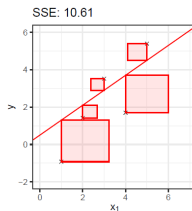
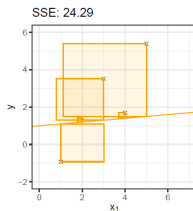
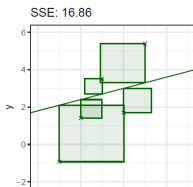
The popular loss function is **L2 loss**, also known as **quadratic loss**:

$$\mathcal{L}(y, f(x)) = (y - f(x))^2$$

Optimization

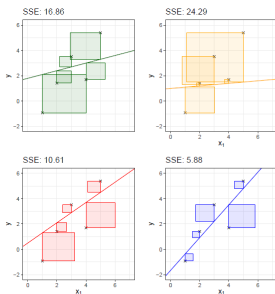
The resulting risk is equivalent to the **sum of squared errors (SSE)**:

$$R(\theta) = \sum_{i=1}^n (y^i - \theta^\top x^i)^2$$

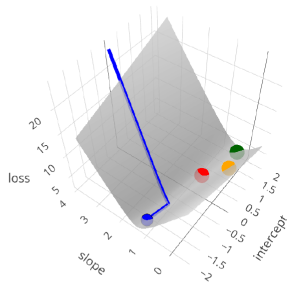


Optimization |

Instead of guessing, of course, use **optimization**!



Intercept θ_0	Slope θ_1	SSE
1.80	0.30	16.86
1.00	0.10	24.29
0.50	0.80	10.61
-1.65	1.29	5.88



Ordinary Least Squares (Component-wise) |

Given data points $(x_1^i, \dots, x_d^i, y^i)$, the model is

$$\hat{y}^i = \sum_{j=1}^d \theta_j x_j^i.$$

The objective is

$$\hat{\theta} \in \operatorname{argmin}_{\theta_1, \dots, \theta_d} R(\theta) = \sum_{i=1}^n \left(y^i - \sum_{j=1}^d \theta_j x_j^i \right)^2.$$

Find by setting partial derivatives to zero:

$$\frac{\partial R(\theta)}{\partial \theta_j} = 0, \quad j = 1, \dots, d.$$

Deriving the Normal Equations (Component-wise) |

Start from

$$R(\theta) = \sum_{i=1}^n \left(y^i - \sum_{k=1}^d \theta_k x_k^i \right)^2.$$

Take partial derivative w.r.t. θ_j :

$$\frac{\partial R}{\partial \theta_j} = -2 \sum_{i=1}^n x_j^i \left(y^i - \sum_{k=1}^d \theta_k x_k^i \right).$$

Set to zero:

$$\sum_{i=1}^n x_j^i y^i = \sum_{i=1}^n x_j^i \sum_{k=1}^d \theta_k x_k^i, \quad j = 1, \dots, d.$$

These d equations determine the optimal parameters $\hat{\theta}_1, \dots, \hat{\theta}_d$.

Optimization – Ordinary Least Squares |

$$\hat{\theta} \in \operatorname{argmin}_{\theta} R(\theta) = \operatorname{argmin}_{\theta} \sum_{i=1}^n (y^i - \theta^{\top} x^i)^2 = \operatorname{argmin}_{\theta} \|y - X\theta\|_2^2$$

Find via **normal equations**:

$$\frac{\partial R(\theta)}{\partial \theta} = 0$$

Solution: Ordinary Least Squares (OLS) Estimator

$$\hat{\theta} = (X^{\top} X)^{-1} X^{\top} y$$

Derive the Normal Equations |

Start from the L2 loss (Square of Euclidean Distance):

$$R(\theta) = \|y - X\theta\|_2^2$$

Expand the quadratic form:

$$R(\theta) = (y - X\theta)^\top (y - X\theta)$$

Take derivative w.r.t. θ :

$$\frac{\partial R(\theta)}{\partial \theta} = -2X^\top (y - X\theta)$$

Set gradient to zero (first-order optimality condition):

$$X^\top X\theta = X^\top y$$

Solve the linear system

$$\hat{\theta} = (X^\top X)^{-1} X^\top y$$

Example: Predicting Coffee Shop Revenue |

A small coffee shop wants a **simple linear model**

$$y = \theta_0 + \theta_1 x$$

to predict daily revenue y from the number of cups sold x . Data:

$$(x, y) \in \{(1, 2), (2, 4), (3, 6)\}.$$

Design matrixes:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \quad X^\top = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}, \quad X^\top X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

$$(X^\top X)^{-1} = \frac{1}{3 \cdot 14 - 6 \cdot 6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}$$

Computing $\hat{\theta}$ Step by Step |

Compute

$$X^{\top}y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} = \begin{bmatrix} 12 \\ 28 \end{bmatrix}.$$

Then

$$(X^{\top}X)^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}.$$

Finally,

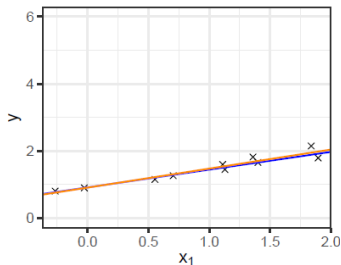
$$\hat{\theta} = (X^{\top}X)^{-1}X^{\top}y = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} \begin{bmatrix} 12 \\ 28 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

The fitted model is

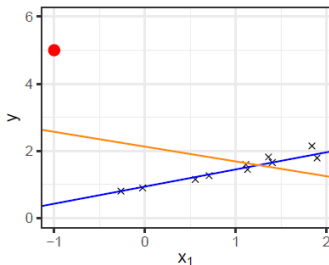
$$\hat{y} = 2x.$$

L1 vs L2 – Robustness |

- L2 loss is **quadratic** in residuals, so outlying points carry lots of weight.
- L1 loss is **more robust** in the presence of outliers.



absolute quadratic



absolute quadratic

Gradient Descent (Not required!) |

Model $f_{w,b}(x) = wx + b$

Parameters w, b

Cost Function $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$

Objective $\underset{w,b}{\text{minimize}} J(w, b)$

Outline:

Start with some w, b (set $w=0, b=0$)

Keep changing w, b to reduce $J(w, b)$

Until we settle at or near a minimum

Gradient Descent (Not required!) – continued |

Gradient Descent

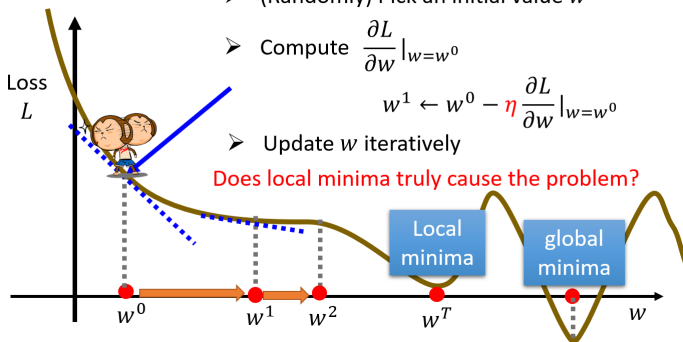
➤ (Randomly) Pick an initial value w^0

➤ Compute $\frac{\partial L}{\partial w} \big|_{w=w^0}$

$$w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w} \big|_{w=w^0}$$

➤ Update w iteratively

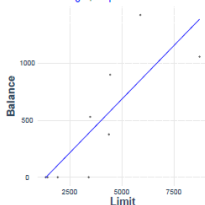
Does local minima truly cause the problem?



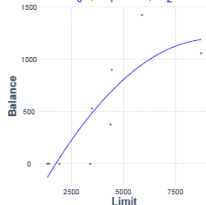
Polynomial Regression |

- Adding **polynomial terms** to the linear combination leads to more flexible regression functions.
- Too high degrees can lead to **overfitting**.

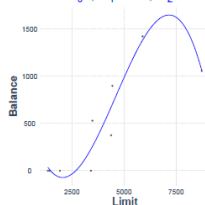
$$\text{Balance} = \theta_0 + \theta_1 \text{Limit}$$



$$\text{Balance} = \theta_0 + \theta_1 \text{Limit} + \theta_2 \text{Limit}^2$$



$$\text{Balance} = \theta_0 + \theta_1 \text{Limit} + \theta_2 \text{Limit}^2 + \theta_3 \text{Limit}^3$$



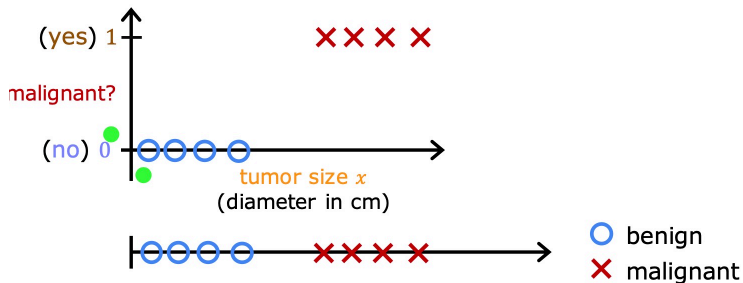
Classification |

Question	No	Yes
Is this email spam?	No	Yes
Is the transaction fraudulent?	No	Yes
Is the tumor malignant?	No	Yes

y can only be one of two values:

- **Binary classification:** $\mathcal{Y} = \{0, 1\}$, $\mathcal{Y} = \{-1, +1\}$,
 $\mathcal{Y} = \{\text{No}, \text{Yes}\}$
- **Multi-class classification:** $\mathcal{Y} = \{1, \dots, g\}$, $g \geq 3$

Classification – Illustration |

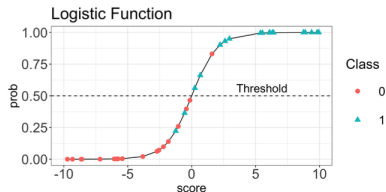


Logistic Regression |

- Logistic regression is a **binary classification** model.
- It turns scores into **probabilities** with the *logistic function*.
- Compute the probability for one class (usually class 1).
- If the probability exceeds a threshold value $c \Rightarrow$ class 1 is predicted.



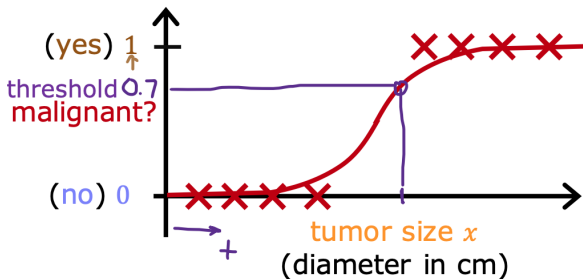
The logistic function puts all scores in order along an s-shaped line.



Logistic Regression – The Logistic Function |

$$g(z) = \frac{1}{1 + e^{-z}}, \quad 0 < g(z) < 1$$
$$z = \theta_0 + \theta_1 x$$

- Output is between 0 and 1 (Sigmoid Function).
- Threshold $c = 0.7$ (example).



Naive Bayes |

Overview

- **Multi-class** classifier.
- **Probabilistic** classification algorithm.
- Based on **Bayes' theorem** with “naive” independence assumptions.

Key Idea

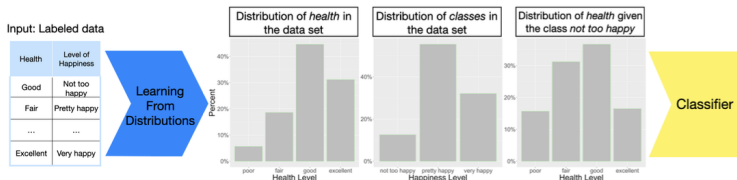
“What’s the probability this instance belongs to class Y given these features?”

Particularly effective for:

- Text classification
- High-dimensional datasets
- Quick baseline models

Bayes' Theorem Foundation |

$$P(Y | X) = \frac{P(X | Y) P(Y)}{P(X)}$$



$P(Y | X)$: **Posterior probability**

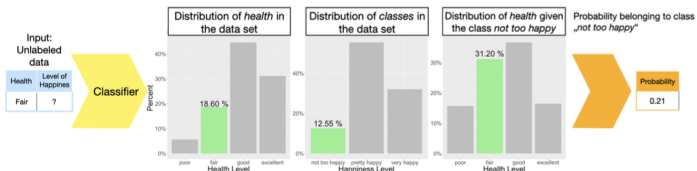
$P(X | Y)$: **Likelihood** – conditional distribution of each feature in each class

$P(Y)$: **Prior probability** – marginal distribution of classes in the entire dataset

$P(X)$: **Evidence** – marginal distribution of each feature in the entire dataset

Naive Bayes – Example |

Example: Class probability of “not too happy” given health = “fair”:



Naive Bayes Classifier

$$\frac{31.20 \% \times 12.55 \%}{18.60 \%} = \mathbb{P}(X = x|Y = k) \times \mathbb{P}(Y = k)$$

Class probability given the data

21.00 %

Naive Bayes Example: Email Spam Detection |

We want to classify an email as **Spam** or **Not Spam** based on two features:

- x_1 : contains the word “free” (yes/no)
- x_2 : contains the word “win” (yes/no)

Training data (4 emails):

Email	x_1 (free)	x_2 (win)	Class
1	yes	yes	Spam
2	yes	no	Spam
3	no	yes	Not Spam
4	no	no	Not Spam

Prior probabilities:

$$P(\text{Spam}) = \frac{2}{4} = 0.5, \quad P(\text{Not Spam}) = 0.5.$$

Task: classify a new email with $x_1 = \text{yes}$, $x_2 = \text{yes}$

Naive Bayes Solution |

Estimate likelihoods from the training data:

$$P(x_1 = \text{yes} \mid \text{Spam}) = \frac{2}{2} \quad P(x_2 = \text{yes} \mid \text{Spam}) = \frac{1}{2}$$

$$P(x_1 = \text{yes} \mid \text{Not Spam}) = \frac{0}{2}, \quad P(x_2 = \text{yes} \mid \text{Not Spam}) = \frac{1}{2}$$

Naive Bayes assumption:

$$P(x_1, x_2 \mid Y) = P(x_1 \mid Y) P(x_2 \mid Y).$$

Compute unnormalized posteriors (proportional to):

$$P(\text{Spam} \mid x_1 = \text{yes}, x_2 = \text{yes}) \propto 1 \times 0.5 \times 0.5 = 0.25,$$

$$P(\text{Not Spam} \mid x_1 = \text{yes}, x_2 = \text{yes}) \propto 0 \times 0.5 \times 0.5 = 0.$$

Therefore, the classifier predicts: **Spam**

What Does $P(X)$ Mean in Naive Bayes? |

In Bayes' rule,

$$P(Y | X) = \frac{P(X | Y) P(Y)}{P(X)},$$

the term $P(x)$ is the **total probability of observing the features X** , regardless of class.

Interpretation:

- $P(X)$ measures how likely the feature pattern X is in the whole population.
- It acts as a **normalizing constant**, all posteriors sum to 1.
- In classification, $P(X)$ is the same for all classes, so it **does not affect which class has the larger posterior**.

Example (from our dataset):

$$P(X) = P(X | \text{Spam})P(\text{Spam}) + P(X | \text{Not Spam})P(\text{Not Spam}) = 0.25$$

$P(X)$ is meaningful, but **unnecessary for classification**.

Naive Bayes – Pros and Cons |

Advantages

- Simple and fast
- Works well with high dimensions
- Requires small training data
- Handles continuous & discrete data

Limitations

- Naive independence assumption
- Sensitive to irrelevant features
- Probability estimates not always accurate

Naive Bayes – Applications & Implementation |

Real-World Applications

- Email spam detection
- Document categorization
- Sentiment analysis
- Medical diagnosis
- Recommender systems

Implementation Tips

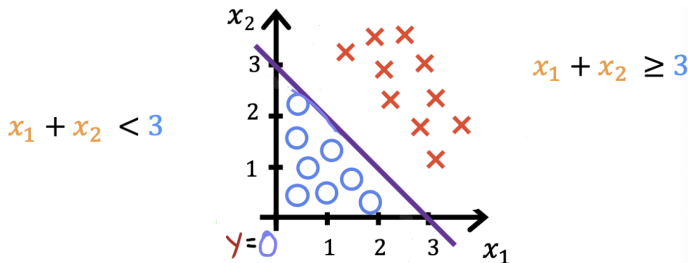
- Handle continuous features with:
 - ▶ Gaussian Naive Bayes
 - ▶ Discretization
- Use **log probabilities** to avoid underflow
- Apply feature selection
- Try different smoothing parameters

Decision Boundary |

Definition

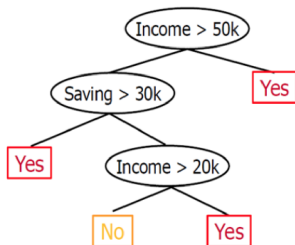
A **decision boundary** is a hypersurface that separates instances of different classes in a classification problem.

- **Linear boundary:** A straight line (in 2D) or a hyperplane (in higher dimensions)
- **Nonlinear boundary**



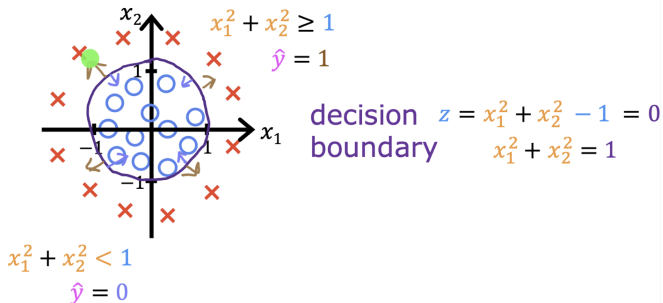
Decision Boundary – Decision Trees |

Decision tree: A series of splits along the feature axes, dividing the feature space into **rectangular regions**.



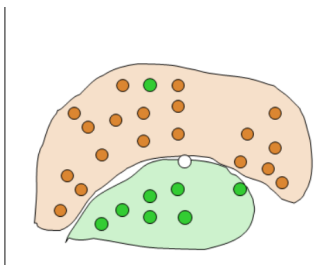
Decision Boundary – Nonlinear |

Nonlinear decision boundary

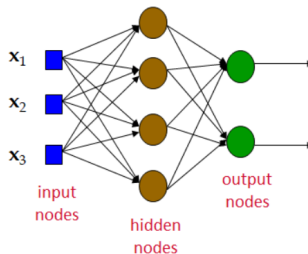


Decision Boundary – Nonlinear (cont.) |

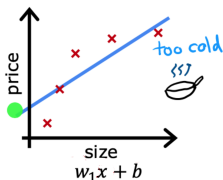
Nonlinear decision boundary



A typical NN

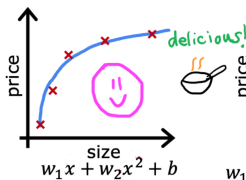


Overfitting and Underfitting – Regression Example |



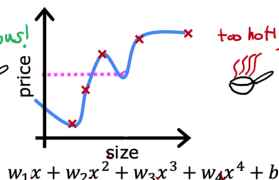
underfit

- Does not fit the training set well



just right

- Fits training set pretty well

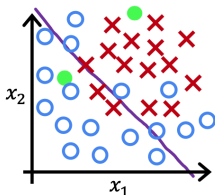


overfit

- Fits the training set extremely well

Overfitting and Underfitting – Classification Example |

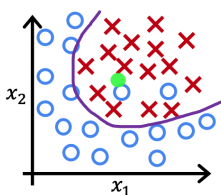
Classification



$$z = w_1 x_1 + w_2 x_2 + b$$

$$f_{\vec{w}, b}(\vec{x}) = g(z)$$

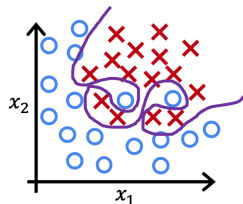
g is the sigmoid function



$$z = w_1 x_1 + w_2 x_2$$

$$+ w_3 x_1^2 + w_4 x_2^2$$

$$+ w_5 x_1 x_2 + b$$



$$z = w_1 x_1 + w_2 x_2$$

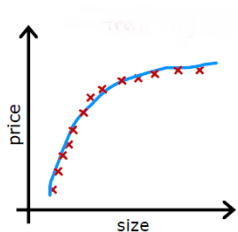
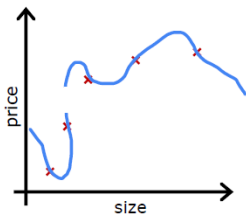
$$+ w_3 x_1^2 x_2 + w_4 x_1^2 x_2^2$$

$$+ w_5 x_1^2 x_2^3 + w_6 x_1^3 x_2$$

$$+ \dots + b$$

Addressing Overfitting |

Strategy 1: Collect more training examples



Addressing Overfitting |

Strategy 2: Select features to include / exclude

size	bedrooms	floors	age	income	...	distance
x_1	x_2	x_3	x_4	x_5	$\dots$	x_{100}

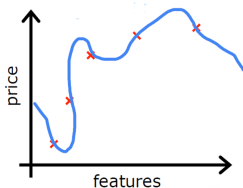
- Choose a **subset of features** most relevant to the prediction target.
- Reducing irrelevant features helps the model generalise better.

Regularization |

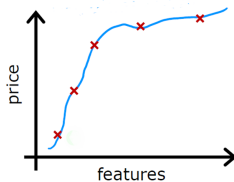
Strategy 3: Reduce the size of parameters

Regularization adds a **penalty term** to the loss function to discourage large parameter values:

$$R_{\text{reg}}(\theta) = \underbrace{\sum_{i=1}^n (y^i - \theta^\top x^i)^2}_{\text{data fit}} + \underbrace{\lambda \|\theta\|^2}_{\text{regularisation}}$$



$$f(x) = 28x - 385x^2 + 39x^3 - 174x^4 + 100$$



$$f(x) = 13x - 0.23x^2 + 0.000014x^3 - 0.0001x^4 + 10$$

What's Coming Next? |

Next Topic

Artificial Neural Network